

Level 1: Dasar AWS Lambda (Konsol & Manual)

Tujuan: Memahami cara kerja fungsi serverless, *trigger*, dan logging tanpa kerumitan Infrastructure as Code (IaC).

Praktikum 1.1: Membuat Fungsi Lambda Pertama

1. Buka konsol AWS Lambda > pilih **Create function**.
2. Pilih **Author from scratch**:
 - o **Function name:** lks-hello-world
 - o **Runtime:** Python 3.12 (atau Node.js 20.x)
3. Tulis kode berikut di editor konsol:

```
Python
import json

def lambda_handler(event, context):
    name = event.get('name', 'Dunia')
    message = f"Halo {name}, selamat datang di AWS Lambda!"

    return {
        'statusCode': 200,
        'body': json.dumps({'message': message})
    }
```

4. Klik **Deploy**.
5. **Pengujian**:
 - o Buat *Test Event* JSON: {"name": "Budi"}
 - o Klik **Test** dan amati hasilnya (statusCode: 200).
 - o Buka **Amazon CloudWatch Logs** untuk melihat log eksekusi (*START, END, REPORT*).

Level 2: Dasar AWS CloudFormation (Infrastructure as Code)

Tujuan: Memahami struktur dasar file YAML/JSON pada CloudFormation untuk membuat infrastruktur secara otomatis.

Praktikum 2.1: Menulis Template CloudFormation Pertama

Buat file bernama 01-basic-s3.yaml:

YAML

AWSTemplateFormatVersion: '2010-09-09'

Description: 'Modul 2.1 - Membuat S3 Bucket Sederhana'

Resources:

MySimpleBucket:

Type: 'AWS::S3::Bucket'

Properties:

BucketName: !Sub 'lks-learning-bucket-\${AWS::AccountId}'

VersioningConfiguration:

Status: Enabled

Outputs:

BucketARN:

Description: 'ARN dari S3 Bucket yang dibuat'

Value: !GetAtt MySimpleBucket.Arn

Langkah Eksekusi:

1. Buka konsol **CloudFormation** > **Create Stack** > **With new resources (standard)**.
2. Unggah file 01-basic-s3.yaml.
3. Masukkan Stack name: Stack-S3-Dasar.
4. Jalankan hingga status menjadi CREATE_COMPLETE. Lihat menu **Outputs** untuk melihat ARN bucket.

Level 3: Menengah (Integrasi Lambda & CloudFormation)

Tujuan: Mendeploy fungsi Lambda beserta IAM Role dan pemicu (*trigger*) S3 secara penuh menggunakan satu template CloudFormation.

Praktikum 3.1: Deploy Serverless Architecture via CloudFormation

Buat file 02-lambda-s3-trigger.yaml:

YAML

AWSTemplateFormatVersion: '2010-09-09'

Description: 'Modul 3.1 - Automation Lambda dan S3 Trigger'

Resources:

1. IAM Role untuk Lambda

LambdaExecutionRole:

Type: 'AWS::IAM::Role'

Properties:

AssumeRolePolicyDocument:

Version: '2012-10-17'

Statement:

- Effect: Allow

Principal:

Service: [lambda.amazonaws.com]

Action: ['sts:AssumeRole']

ManagedPolicyArns:

- 'arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole'

2. Fungsi Lambda (In-line Code)

S3ProcessorLambda:

Type: 'AWS::Lambda::Function'

Properties:

FunctionName: 'lks-s3-event-processor'

Handler: 'index.lambda_handler'

Role: !GetAtt LambdaExecutionRole.Arn

Runtime: 'python3.12'

Timeout: 10

Code:

ZipFile: |

import json

def lambda_handler(event, context):

for record in event['Records']:

bucket = record['s3']['bucket']['name']

key = record['s3']['object']['key']

print(f"File baru diunggah: {key} pada bucket: {bucket}")

return {'statusCode': 200}

3. Izin S3 memanggil Lambda

LambdaPermissionForS3:

Type: 'AWS::Lambda::Permission'

Properties:

Action: 'lambda:InvokeFunction'

FunctionName: !Ref S3ProcessorLambda

```
Principal: 's3.amazonaws.com'
```

```
SourceArn: !GetAtt SourceBucket.Arn
```

4. S3 Bucket dengan Event Notification

```
SourceBucket:
```

```
Type: 'AWS::S3::Bucket'
```

```
Properties:
```

```
BucketName: !Sub 'lks-upload-trigger-${AWS::AccountId}'
```

```
NotificationConfiguration:
```

```
LambdaConfigurations:
```

```
- Event: 's3:ObjectCreated:*'
```

```
Function: !GetAtt S3ProcessorLambda.Arn
```

```
Outputs:
```

```
BucketName:
```

```
Value: !Ref SourceBucket
```

```
LambdaARN:
```

```
Value: !GetAtt S3ProcessorLambda.Arn
```

Level 4: Advanced (Infrastruktur Kompleks & Parameterization)

Tujuan: Menggunakan fitur lanjutan CloudFormation (Parameters, Mappings, Conditionals) untuk mendeploy arsitektur REST API terintegrasi dengan Lambda dan DynamoDB.

Praktikum 4.1: Serverless Microservice (API Gateway + Lambda + DynamoDB)

Buat file 03-advanced-microservice.yaml:

```
YAML
```

```
AWSTemplateFormatVersion: '2010-09-09'
```

```
Description: 'Modul 4.1 - Microservice CRUD Serverless Kompleks'
```

```
Parameters:
```

```
EnvironmentType:
```

```
Type: String
```

```
Default: dev
```

```
AllowedValues: [dev, prod]
```

```
Description: 'Pilih lingkungan deployment'
```

Conditions:

IsProduction: !Equals [!Ref EnvironmentType, 'prod']

Resources:

Database Layer

UsersTable:

Type: 'AWS::DynamoDB::Table'

Properties:

TableName: !Sub 'UsersTable-\${EnvironmentType}'

BillingMode: PAY_PER_REQUEST

AttributeDefinitions:

- AttributeName: UserId

AttributeType: S

KeySchema:

- AttributeName: UserId

KeyType: HASH

Compute Layer

ApiLambdaRole:

Type: 'AWS::IAM::Role'

Properties:

AssumeRolePolicyDocument:

Version: '2012-10-17'

Statement:

- Effect: Allow

Principal:

Service: [lambda.amazonaws.com]

Action: ['sts:AssumeRole']

Policies:

- PolicyName: DynamoDBAccess

PolicyDocument:

Version: '2012-10-17'

Statement:

- Effect: Allow

Action:

- 'dynamodb:PutItem'

- 'dynamodb:GetItem'

Resource: !GetAtt UsersTable.Arn

- Effect: Allow

Action:

- 'logs:CreateLogGroup'

- 'logs:CreateLogStream'

- 'logs:PutLogEvents'

Resource: '*'

UserApiLambda:

Type: 'AWS::Lambda::Function'

Properties:

FunctionName: !Sub 'user-service-\${EnvironmentType}'

Handler: 'index.handler'

Role: !GetAtt ApiLambdaRole.Arn

Runtime: 'python3.12'

MemorySize: !If [IsProduction, 512, 128]

Environment:

Variables:

TABLE_NAME: !Ref UsersTable

Code:

ZipFile: |

import json, boto3, os

dynamodb = boto3.resource('dynamodb')

table = dynamodb.Table(os.environ['TABLE_NAME'])

def handler(event, context):

body = json.loads(event.get('body', '{}'))

table.put_item(Item={'UserId': body['id'], 'Name': body['name']})

return {

'statusCode': 200,

'body': json.dumps({'message': 'User berhasil disimpan'})

}

API Gateway Layer

RestApi:

Type: 'AWS::ApiGateway::RestApi'

Properties:

Name: !Sub 'UserAPI-\${EnvironmentType}'

ApiResource:

Type: 'AWS::ApiGateway::Resource'

Properties:

ParentId: !GetAtt RestApi.RootResourceId

PathPart: 'user'

RestApiId: !Ref RestApi

ApiMethod:

Type: 'AWS::ApiGateway::Method'

Properties:

RestApiId: !Ref RestApi

```
ResourceId: !Ref ApiResource
HttpMethod: 'POST'
AuthorizationType: 'NONE'
Integration:
  Type: 'AWS_PROXY'
  IntegrationHttpMethod: 'POST'
  Uri: !Sub 'arn:aws:apigateway:${AWS::Region}:lambda:path/2015-03-31/functions/${UserApiLambda.Arn}/invocations'
```

```
LambdaApiPermission:
  Type: 'AWS::Lambda::Permission'
  Properties:
    Action: 'lambda:InvokeFunction'
    FunctionName: !Ref UserApiLambda
    Principal: 'apigateway.amazonaws.com'
```

```
ApiDeployment:
  Type: 'AWS::ApiGateway::Deployment'
  DependsOn: ApiMethod
  Properties:
    RestApiId: !Ref RestApi
    StageName: !Ref EnvironmentType
```

```
Outputs:
  ApiEndpoint:
    Description: 'URL HTTP POST Endpoint'
    Value: !Sub 'https://${RestApi}.execute-api.${AWS::Region}.amazonaws.com/${EnvironmentType}/user'
```

Panduan Alur Belajar Ringkas

1. Praktikum 1.1 & 2.1. Pahami apa itu *Resource*, *Handler*, *IAM Role*, dan *CloudWatch Logs*.
2. Praktikum 3.1. Pelajari bagaimana CloudFormation menghubungkan resource satu sama lain menggunakan `!Ref`, `!GetAtt`, serta konsep S3 Event Notification.
3. Praktikum 4.1. Pelajari penggunaan parameter, logika `!If`, penyusunan lingkungan dev/prod, dan integrasi API Gateway.